چالش ۳۰ روزهٔ اندروید

آمادگی مصاحبهٔ اندروید — جونیور ← مید ← سینیور ← آرکیتکت

نسخهٔ ۲۰۲۶

روز ۳

الگوهای معماری — از MVC تا MVVM و MVI، همراه با UDF و Clean Architecture



اسماعیل مرادی

@esmaeeil-moradi700

چرا معماری روز سوم است؟

سؤال‌های معماری به مصاحبه‌گر نشان می‌دهند **چطور فکر می‌کنید**، نه اینکه چه کتابخانه‌هایی بلدید. جواب ۲۰۲۶ دیگر «MVP یا MVVM» نیست:

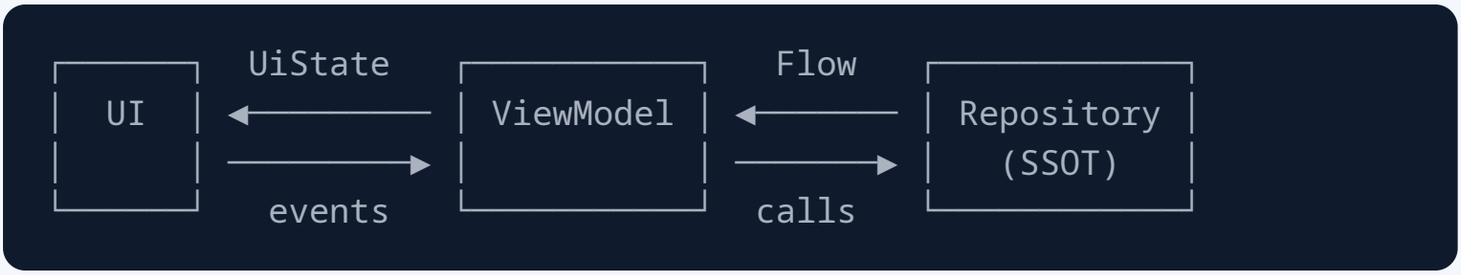
- راهنمای معماری گوگل: **لایهٔ UI، لایهٔ domain (اختیاری) و لایهٔ data**
- **جریان دادهٔ یک‌طرفه (UDF)** و **منبع واحد حقیقت (SSOT)**
- MVVM با یک `UiState` تغییرناپذیر — که همین حالا بیشتر MVI را در خودش دارد
- ایده‌های Clean Architecture، به شکل **عمل‌گرایانه**
- همین معماری، مشترک بین پلتفرم‌ها با KMP

سطح	سؤال‌ها
جونیور	MVC و MVP و MVVM، چرا ViewModel، لایه‌ها
مید	UDF و SSOT، مدل‌سازی UI state، eventهای یک‌باره، MVVM یا MVI
سینیور	Repository به‌عنوان SSOT، use caseها، تست معماری
آرکیتکت	Clean Architecture درست، حاکمیت معماری با کد، KMP



اسماعیل مرادی
@esmaeeil-moradi700

مدل ذهنی: state به پایین، event به بالا



- **state به پایین می‌آید:** UI فقط چیزی را که به او داده شده رندر می‌کند
- **event به بالا می‌رود:** UI گزارش می‌دهد چه اتفاقی افتاد، خودش هیچ‌وقت state را تغییر نمی‌دهد
- **منبع واحد حقیقت:** هر داده دقیقاً یک مالک دارد که اجازهٔ تغییرش را دارد

ایدهٔ واحد پشت همهٔ الگوهای امروز

اسمش هر چه باشد — MVVM، MVI، Clean — جواب خوب همیشه به این برمی‌گردد: *مالک این state کیست، و داده در چه جهتی جریان دارد؟*



اسماعیل مرادی
@esmaeeil-moradi700

جونیور

MVC و MVP و MVVM، چرا ViewModel، لایه‌ها



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱. MVC، MVP و MVVM — هر کدام چه مشکلی را حل می‌کنند

الگو	منطق نمایش کجاست	view چطور به‌روز می‌شود	نقطه‌ضعف در اندروید
MVC	در Controller — که معمولاً خود Activity است	Controller مستقیم view را تغییر می‌دهد	همان God Activity معروف: تست‌ناپذیر و با چرخش صفحه از بین می‌رود
MVP	در Presenter، پشت یک interface به اسم `View`	Presenter متد `view.showX()` را صدا می‌زند	از چرخش صفحه جان سالم به در نمی‌برد؛ attach و detach دستی
MVVM	در ViewModel که state قابل‌مشاهده منتشر می‌کند	view یک `StateFlow` را observe می‌کند	بدون نظم: streamهای زیاد و state پراکنده

هر سه، UI را از منطق جدا می‌کنند. تفاوت در این است که **چه کسی چه کسی را می‌شناسد**: presenter ارجاعی به view نگه می‌دارد؛ ViewModel **هیچ چیزی** دربارهٔ view نمی‌داند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱. چیزهایی که مصاحبه‌گر چک می‌کند

تلهٔ مصاحبه

«*MVVM یعنی Data Binding.*» — **نه.** Data Binding فقط یک راه برای observe کردن ViewModel بود. در ۲۰۲۶ لایهٔ view یا Compose است، یا Viewهایی که با `repeatOnLifecycle` یک `StateFlow` را collect می‌کنند.

تلهٔ مصاحبه

«*در MVC، اکتیویتی همان View است.*» — در اندروید، Activity در عمل هم View می‌شود **و هم** Controller. God Activity دقیقاً همین‌طور به دنیا می‌آید.

سؤال تکمیلی

«*چرا اندروید از MVP به MVVM رفت؟*» — `ViewModel` در Jetpack از configuration change عبور می‌کند و `viewModelScope` دارد. presenter هیچ‌کدام را ندارد، برای همین هر پروژهٔ MVP منطق نگه‌داری و attach/detach را از نو اختراع می‌کرد.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۲. چرا ViewModel به اندروید می‌خورد

```
class ProfileViewModel(
    repo: ProfileRepository,
    handle: SavedStateHandle,
) : ViewModel() {
    private val userId: String = checkNotNull(handle["userId"])

    val uiState: StateFlow<ProfileUiState> =
        repo.observe(userId)
            .map { ProfileUiState.Ready(it) }
            .stateIn(
                scope = viewModelScope,
                started = SharingStarted.WhileSubscribed(5_000),
                initialValue = ProfileUiState.Loading,
            )
}
```

- از **configuration change** عبور می‌کند؛ فقط وقتی مالکش برای همیشه از بین برود پاک می‌شود (`onCleared()` صدا زده و `viewModelScope` لغو می‌شود)
- به یک مالک scope دارد: Activity، Fragment یا یک back-stack entry در navigation



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۲. محدودیت‌هایی که باید بدانید

- ViewModel **از process death عبور نمی‌کند.** کلیدهای لازم برای بازسازی صفحه در `SavedStateHandle` می‌روند؛ خود داده از repository برمی‌گردد.
- ViewModel **state و منطق** نگه می‌دارد، هرگز view.

تلهٔ مصاحبه

نگه داشتن `Context`، `View` یا `Activity` در ViewModel، کل صفحهٔ قبلی را بعد از چرخش نشت می‌دهد. به resource یا سرویس سیستمی نیاز دارید؟ آن را پشت یک repository بگذارید و inject کنید. `AndroidViewModel` معمولاً نشانهٔ یک طراحی بد است.

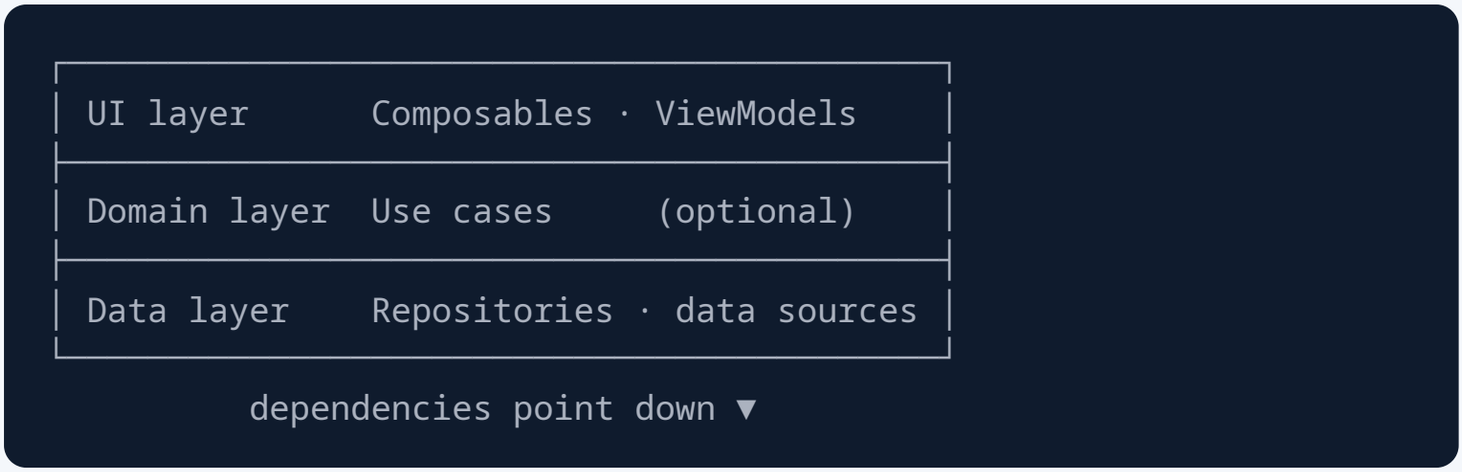
سؤال تکمیلی

«*چه کسی ViewModel را با پارامترهای constructor می‌سازد؟*» — یک `ViewModelProvider.Factory`؛ که در عمل یا DI (مثل Hilt یا Koin) آن را می‌سازد یا DSL مربوط به `viewModelFactory { initializer { … } }`.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۳. لایه‌های یک اپ اندروید مدرن



- **لایهٔ UI:** state را رندر می‌کند و event می‌فرستد. state holderها (ViewModelها) داده را به UI state تبدیل می‌کنند.
- **لایهٔ domain (اختیاری):** منطق بیزینسی که چند ViewModel از آن استفاده می‌کنند.
- **لایهٔ data:** repositoryها داده را منتشر می‌کنند و قواعدش را در اختیار دارند؛ هر data source فقط با یک منبع کار می‌کند (Room، Ktor/Retrofit، DataStore).

تلهٔ مصاحبه

«*هر اپی به لایهٔ domain نیاز دارد.*» — در راهنمای گوگل این لایه **اختیاری** است. وقتی اضافه‌اش کنید که منطق چند جا استفاده می‌شود یا آن‌قدر پیچیده است که تست جداگانه بخواهد.



اسماعیل مرادی
@esmaeeil-moradi700

مید

UDF و SSOT، مدل‌سازی UI state، eventهای یک‌باره، MVVM یا MVI



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۴. UDF و SSOT در کد

```
data class SearchUiState(
    val query: String = "",
    val results: List<Item> = emptyList(),
    val isLoading: Boolean = false,
)

class SearchViewModel : ViewModel() {
    private val _uiState = MutableStateFlow(SearchUiState())
    val uiState: StateFlow<SearchUiState> = _uiState.asStateFlow()

    fun onQueryChange(query: String) {
        _uiState.update { it.copy(query = query) }
    }
}
```

- فقط ViewModel state را **می‌نویسد**؛ بقیه یک `StateFlow` فقط‌خواندنی را می‌خوانند
- `update {}` اتمیک است (compare-and-set) — وقتی چند coroutine هم‌زمان state را تغییر می‌دهند امن است



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۴. سمت UI — یک route با state و یک screen بدون state

```
@Composable
fun SearchRoute(viewModel: SearchViewModel = hiltViewModel()) {
    val state by viewModel.uiState.collectAsStateWithLifecycle()
    SearchScreen(state, onQueryChange = viewModel::onQueryChange)
}

@Composable
fun SearchScreen(
    state: SearchUiState,
    onQueryChange: (String) -> Unit,
) { /* stateless: easy to preview and test */ }
```

- **route** به ViewModel وصل می‌شود؛ **screen** یک تابع خالص از پارامترهایش است
- `collectAsStateWithLifecycle()` وقتی اپ به background می‌رود collect را متوقف می‌کند

تلهٔ مصاحبه

منتشر کردن عمومی `MutableStateFlow` : حالا هر صفحه‌ای می‌تواند آن را بنویسد و دیگر منبع واحد حقیقتی وجود ندارد.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۵. مدل‌سازی UI state

sealed interface — وقتی حالت‌ها با هم جمع نمی‌شوند:

```
sealed interface FeedUiState {
    data object Loading : FeedUiState
    data class Success(val posts: List<Post>) : FeedUiState
    data class Error(val message: String) : FeedUiState
}
```

data class — وقتی بخش‌ها کنار هم وجود دارند (محتوا، در حال refresh، پیام):

```
data class FeedUiState(
    val posts: List<Post> = emptyList(),
    val isRefreshing: Boolean = false,
    val userMessage: String? = null,
)
```

حالت‌های غیرممکن را غیرممکن کنید

اگر «در حال بارگذاری» و «خطا» هیچ‌وقت هم‌زمان رخ نمی‌دهند، آن‌ها را با دو boolean که هر دو می‌توانند `true` باشند مدل نکنید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۵. یک stream برای هر صفحه

```
val uiState: StateFlow<FeedUiState> = combine(
    repo.posts, repo.isRefreshing, userMessages,
) { posts, refreshing, message ->
    FeedUiState(posts, refreshing, message)
}.stateIn(
    scope = viewModelScope,
    started = SharingStarted.WhileSubscribed(5_000),
    initialValue = FeedUiState(),
)
```

- یک `StateFlow<UiState>` برای هر صفحه: یک مسیر رندر، یک چیز برای تست
- `WhileSubscribed(5_000)`: جریان بالادستی ۵ ثانیه بعد از رفتن UI متوقف می‌شود، اما با چرخش صفحه از نو شروع نمی‌شود
- state تغییرناپذیر (`val` و `List` فقط‌خواندنی) — Compose می‌تواند با خیال راحت recomposition را رد کند

تلهٔ مصاحبه

«یک *state غول‌پیکر برای کل اپ*.» — state را به صفحه محدود کنید. state سراسری اپ جایش در repositoryهاست.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۶. eventهای یک‌باره: ناوبری، snackbar، «پرداخت انجام شد»

روش	ریسک
`SharedFlow(replay = 0)`	اگر همان لحظه کسی collect نکند (چرخش صفحه، background) event **دور ریخته می‌شود**
`Channel` همراه با `receiveAsFlow()`	بافر دارد، اما اگر روی `Dispatchers.Main.immediate` collect نشود باز هم ممکن است گم شود
`SingleLiveEvent`	ترفند قدیمی LiveData؛ فقط یک observer
event به شکل state (توصیهٔ گوگل)	از چرخش صفحه عبور می‌کند؛ UI به ViewModel خبر می‌دهد که event را مدیریت کرد

تلهٔ مصاحبه

«`Channel` *تحویل را تضمین می‌کند.*» — نه بعد از process death، و نه وقتی بیرون از `Main.immediate` collect شود. برای نتیجه‌ای که هرگز نباید گم شود (مثل پرداخت) از **state + ذخیرهٔ پایدار** استفاده کنید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۶. event به شکل state

```
data class CheckoutUiState(
    val isPaying: Boolean = false,
    val paymentResult: PaymentResult? = null,
)

// ViewModel
fun onPaymentResultHandled() {
    _uiState.update { it.copy(paymentResult = null) }
}

// UI
state.paymentResult?.let { result ->
    LaunchedEffect(result) {
        onNavigateToReceipt(result.id)
        viewModel.onPaymentResultHandled()
    }
}
```

نتیجه تا وقتی UI **تأیید نکند** که آن را مدیریت کرده، در state می‌ماند — چرخش صفحه، background یا یک collector کند نمی‌تواند آن را گم کند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۷. MVI در عمل

MVI یعنی MVVM + یک state تغییرناپذیر + همهٔ ورودی‌ها به شکل intent تایپ‌دار + یک reducer خالص.

```
sealed interface CartIntent {
    data class Add(val item: Item) : CartIntent
    data class Remove(val id: String) : CartIntent
    data object Checkout : CartIntent
}

fun CartState.reduce(intent: CartIntent): CartState =
    when (intent) {
        is CartIntent.Add -> copy(items = items + intent.item)
        is CartIntent.Remove ->
            copy(items = items.filter { it.id != intent.id })
        CartIntent.Checkout -> copy(isCheckingOut = true)
    }
```

reducer خالص است: state یکسان و intent یکسان همیشه نتیجهٔ یکسان می‌دهد. side effectها (شبکه، ناوبری) **بیرون** از آن اجرا می‌شوند و به شکل intent جدید برمی‌گردند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۷. MVVM یا MVI؟

	MVVM + UDF	MVI
ورودی	تابع‌ها (`onQueryChange()`)	یک نقطهٔ ورود `onIntent(intent)`
state	یک `UiState` (توصیه‌شده)	یک `UiState` (الزامی)
منطق	داخل تابع‌های ViewModel	reducer خالص + effectهای جدا
مناسب برای	بیشتر صفحه‌ها	صفحه‌های پیچیده و پر از event: ویرایشگرها، ترید، flowهای چندمرحله‌ای
هزینه	کم	تایپ‌های بیشتر، تشریفات بیشتر

نظر من

در ۲۰۲۶ فاصله کم است: MVVM با یک `UiState` و UDF همین حالا بیشتر MVI را دارد. MVI را وقتی انتخاب کنید که به تضمین‌های reducer نیاز دارید — قابلیت ممیزی، بازپخش intentها در تست — نه چون مد است.

تلهٔ مصاحبه

«*MVI به یک فریم‌ورک نیاز دارد.*» — MVI یک نظم است. یک sealed interface و یک `when` کافی است؛ Orbit یا Mavericks اختیاری‌اند.



اسماعیل مرادی
@esmaeeil-moradi700

سینیور

Repository به‌عنوان SSOT، use caseها، تست معماری



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۸. repository به‌عنوان منبع واحد حقیقت

```
class ArticleRepository(
    private val dao: ArticleDao,
    private val api: ArticleApi,
) {
    // The database is the single source of truth
    fun observeArticles(): Flow<List<Article>> =
        dao.observeAll().map { rows -> rows.map { it.toDomain() } }

    suspend fun refresh(): Result<Unit> = try {
        val remote = api.fetchArticles()
        dao.upsertAll(remote.map { it.toEntity() })
        Result.success(Unit)
    } catch (e: CancellationException) {
        throw e
    } catch (e: IOException) {
        Result.failure(e)
    }
}
```

UI **دیتابیس** را observe می‌کند؛ شبکه فقط **در آن می‌نویسد**. یک حقیقت، کار در حالت آفلاین، و دیگر سؤال «کدام نسخه درست است؟» پیش نمی‌آید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۸. قواعد یک repository خوب

- **تبدیل مدل در مرز لایه‌ها:** از DTO به مدل domain و بعد به مدل UI. لایهٔ UI هیچ‌وقت یک `@Entity` یا DTO شبکه نمی‌بیند.
- `Flow` برای streamها، `suspend` برای عملیات یک‌باره. state قابل‌تغییر را **هرگز** منتشر نکنید.
- **main-safe:** صدازننده هیچ‌وقت ترد عوض نمی‌کند. فراخوانی‌های `suspend` در Room و Retrofit/Ktor خودشان main-safe هستند؛ فقط کار سنگین CPU (parse بزرگ، مرتب‌سازی) را به یک dispatcher پس‌زمینه ببرید.

تلهٔ مصاحبه

`runCatching { }` داخل coroutine، `CancellationException` را هم می‌گیرد و لغو ساختاریافته (structured cancellation) را خراب می‌کند. آن را دوباره throw کنید، یا فقط exceptionهای مشخص را بگیرید.

تلهٔ مصاحبه

cache کردن همان داده هم در ViewModel **و هم** در Room — حالا دو منبع حقیقت دارید، و دیر یا زود با هم اختلاف پیدا می‌کنند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۹. use caseها: کِی ارزش جایشان را دارند

```
class GetLatestNewsWithAuthorsUseCase(
    private val news: NewsRepository,
    private val authors: AuthorsRepository,
) {
    operator fun invoke(): Flow<List<ArticleWithAuthor>> =
        combine(news.latest(), authors.byId()) { articles, byId ->
            articles.map { a ->
                ArticleWithAuthor(a, byId.getValue(a.authorId))
            }
        }
}
```

- وقتی اضافه‌اش کنید که منطق را چند ViewModel **استفاده می‌کنند**، یا آن‌قدر **پیچیده** است که تست جداگانه بخواهد
- یک کار، اسم فعلی (`GetLatestNews…`، `ValidateIban…`)، Kotlin خالص، بدون import اندرویدی

تلهٔ مصاحبه

use caseهای واسطهٔ خالی — یک `GetUserUseCase` که فقط `repo.getUser()` را صدا می‌زند. تشریفات بدون ارزش؛ ViewModel می‌تواند مستقیم repository را صدا بزند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۰. تست معماری — آماده‌سازی

```
class MainDispatcherRule(
    val dispatcher: TestDispatcher = UnconfinedTestDispatcher(),
) : TestWatcher() {
    override fun starting(description: Description) {
        Dispatchers.setMain(dispatcher)
    }
    override fun finished(description: Description) {
        Dispatchers.resetMain()
    }
}
```

- `viewModelScope` روی `Dispatchers.Main` اجرا می‌شود — در unit test جایگزینش کنید
- بقیهٔ dispatcherها را inject کنید تا تست بتواند کنترلشان کند
- برای repositoryها **fake به‌جای mock**: رفتار واقعی، بدون زنجیره‌های شکنندهٔ `verify`



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۰. تست یک ViewModel

```
class FeedViewModelTest {
    @get:Rule val mainRule = MainDispatcherRule()

    @Test fun showsPostsFromRepository() = runTest {
        val repo = FakeFeedRepository(posts = listOf(post1))
        val viewModel = FeedViewModel(repo)

        val dispatcher = UnconfinedTestDispatcher(testScheduler)
        backgroundScope.launch(dispatcher) {
            viewModel.uiState.collect()
        }

        val state = viewModel.uiState.value
        assertEquals(listOf(post1), state.posts)
    }
}
```

- `stateIn(WhileSubscribed)` فقط وقتی collector دارد اجرا می‌شود — یکی در `backgroundScope` راه بیندازید
- screen بی‌state را جداگانه با `createComposeRule()` تست کنید

تلهٔ مصاحبه

`Thread.sleep()` یا تأخیر واقعی در تست. از زمان‌بند تست استفاده کنید: `advanceUntilIdle()` و `advanceTimeBy()`.



اسماعیل مرادی
@esmaeeil-moradi700

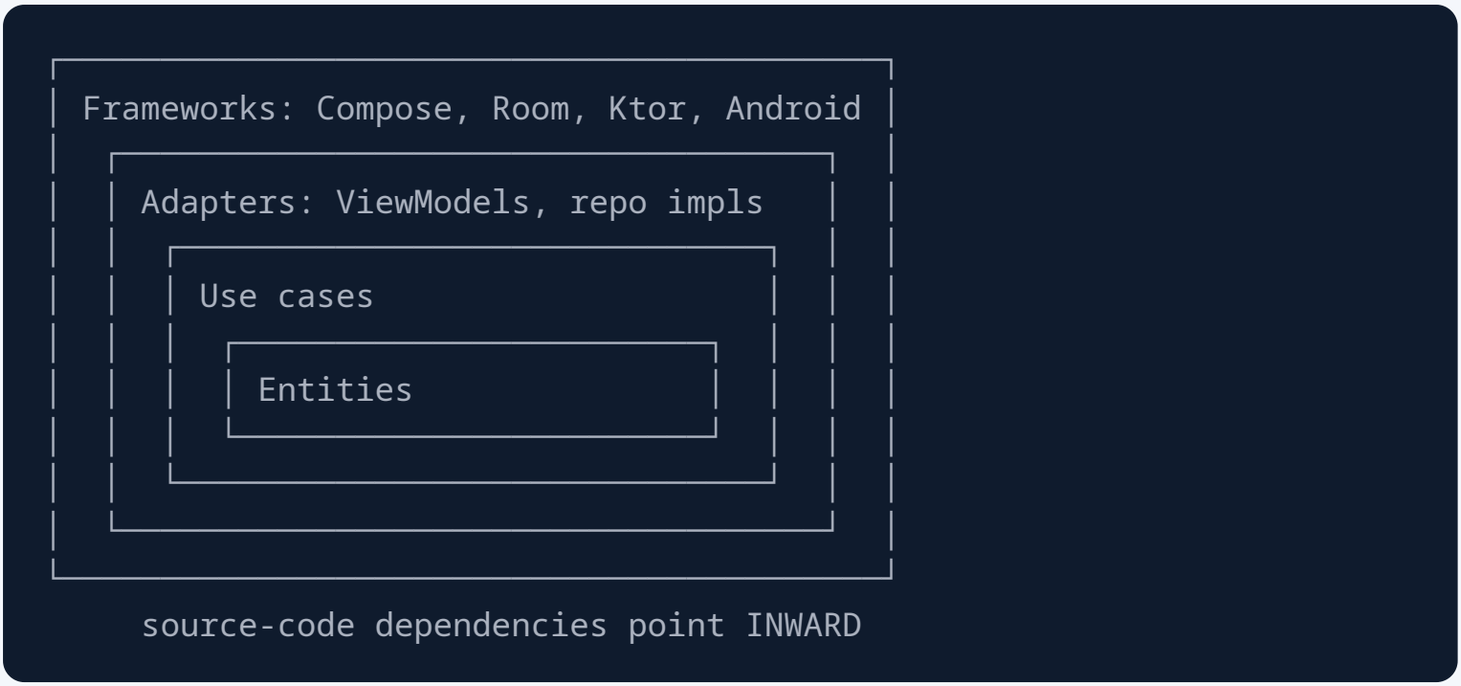
آرکیتکت

Clean Architecture درست، حاکمیت معماری با کد، KMP



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۱. Clean Architecture: قانون وابستگی



- دایره‌های داخلی **هیچ چیزی** دربارهٔ دایره‌های بیرونی نمی‌دانند. لایه‌های بیرونی به لایه‌های داخلی وابسته‌اند — هرگز برعکس.
- **وارونگی وابستگی (Dependency Inversion):** لایهٔ domain یک `interface ArticleRepository` تعریف می‌کند؛ لایهٔ data آن را پیاده می‌کند؛ DI این دو را به هم وصل می‌کند.

تلهٔ مصاحبه

«*لایه‌های بیرونی نباید به لایه‌های داخلی وابسته باشند.*» — برعکس است. این لایه‌های **داخلی** هستند که هرگز نباید به لایه‌های بیرونی وابسته باشند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۱. کجا جواب می‌دهد — و کجا دردسر می‌سازد

جواب می‌دهد	دردسر می‌سازد
تیم‌های بزرگ، محصول بلندمدت، منطق مشترک بین پلتفرم‌ها	اپ‌های کوچک، نمونهٔ اولیه، صفحه‌های CRUD ساده
قواعد بیزینسی که باید جداگانه تست شوند (فین‌تک، سلامت)	تبدیل یک مدل در سه لایه بدون هیچ منطقی در میان
تعویض فریم‌ورک (از Views به Compose، از Retrofit به Ktor)	یک ماژول برای هر لایهٔ هر فیچر، برای پنج صفحه

- ساختار را **اول بر اساس فیچر** بچینید و لایه‌ها را داخل آن (`feature/checkout/{ui, domain, data}`) — نه پوشه‌های سراسری `ui/` و `domain/` و `data/` برای کل اپ.

سؤال تکمیلی

«*یک اپ قدیمی MVP را چطور به این معماری می‌برید؟*» — با رویکرد strangler: فیچرهای جدید در معماری هدف؛ اول repositoryها را جدا کنید (بزرگ‌ترین برد: SSOT و تست‌پذیری)؛ بعد presenterها را صفحه به صفحه با ViewModel عوض کنید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۲. حاکمیت معماری با کد

قاعده‌هایی که فقط در ویکی نوشته شده‌اند کم‌کم فرسوده می‌شوند. آن‌ها را به شکل **fitness function** بنویسید که build در CI را fail کنند:

```
class ArchitectureTest {
    @Test fun layersRespectDependencyRule() {
        Konsist.scopeFromProduction().assertArchitecture {
            val domain = Layer("Domain", "com.app.domain..")
            val data = Layer("Data", "com.app.data..")
            val ui = Layer("UI", "com.app.ui..")

            domain.dependsOnNothing()
            data.dependsOn(domain)
            ui.dependsOn(domain)
        }
    }

    @Test fun viewModelsLiveInUiLayer() {
        Konsist.scopeFromProduction()
            .classes()
            .withNameEndingWith("ViewModel")
            .assertTrue { it.resideInPackage("..ui..") }
    }
}
```



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۲. جعبه‌ابزار حاکمیت معماری

ابزار	چه چیزی را اعمال می‌کند
مرز ماژول‌های Gradle (`api` / `impl`، ترجیح `implementation` به `api`)	چه کسی چه چیزی را می‌بیند — در زمان کامپایل
Konsist / ArchUnit	قواعد لایه، نام‌گذاری و package به شکل unit test
چک‌های سفارشی Android Lint، detekt، ktlint	قواعد سطح کد و سبک نوشتن
Dependency Analysis Gradle Plugin	وابستگی‌های بلااستفاده یا اشتباه تعریف‌شده
ADR داخل ریپو	*چرا* یک تصمیم گرفته شد

نظر من: هستهٔ محکم، لبه‌های منعطف

قواعد سخت‌گیرانه در مرز ماژول‌ها و ماژول‌های هستهٔ مشترک؛ تیم‌ها داخل فیچر خودشان آزادند (برای هر صفحه MVVM یا MVI). ابزارها «پلیس خوب» هستند؛ ADRها و گیلدها *چرایی* را توضیح می‌دهند.

تلهٔ مصاحبه

حاکمیت معماری فقط از طریق code review — مقیاس‌پذیر نیست و آرکیتکت را گلوگاه می‌کند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۳. یک معماری، چند پلتفرم: KMP

```
commonMain   domain · data (Ktor, Room KMP, DataStore)
             ViewModels (androidx.lifecycle KMP)
──────────────────────────────────────────────────────
androidMain  Compose UI    |  iosMain  SwiftUI or CMP
```

- `ViewModel` و `viewModelScope` در Jetpack از Lifecycle 2.8 چندسکویی (multiplatform) شده‌اند؛ Room از نسخهٔ 2.7 از KMP پشتیبانی می‌کند
- اول **data** و **domain** را مشترک کنید (بیشترین ارزش، کمترین ریسک)، بعد ViewModelها؛ UI را (با Compose Multiplatform) فقط وقتی مشترک کنید که به تیم بخورد
- APIهای پلتفرم را پشت interface نگه دارید (`expect` / `actual` یا DI)

تلهٔ مصاحبه

«*KMP یعنی اپ iOS با Kotlin نوشته می‌شود.*» — لزوماً نه. منطق مشترک با UI بومی رایج‌ترین و کم‌ریسک‌ترین حالت است.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۳. یک ViewModel مشترک

```
// commonMain — compiled for Android and iOS
class ArticlesViewModel(
    private val repo: ArticleRepository,
) : ViewModel() {
    val articles: StateFlow<List<Article>> =
        repo.observeArticles().stateIn(
            scope = viewModelScope,
            started = SharingStarted.WhileSubscribed(5_000),
            initialValue = emptyList(),
        )

    fun refresh() {
        viewModelScope.launch { repo.refresh() }
    }
}
```

- **همان** قواعد UDF روی همهٔ پلتفرم‌ها برقرار است: یک state، eventها به بالا، repository به‌عنوان SSOT
- معماری‌ای که روی اندروید انتخاب کردید، قرارداد کل محصول می‌شود

جواب سینیور در یک جمله

«الگو را برای هر صفحه انتخاب کن، اما قواعد را برای کل اپ ثابت کن: یک state برای هر صفحه، جریان یک‌طرفهٔ داده، یک مالک برای هر داده، و وابستگی‌هایی که رو به داخل‌اند.»



اسماعیل مرادی
@esmaeeil-moradi700

روز ۳ — خلاصهٔ یک‌صفحه‌ای

- از MVC تا MVP و MVVM، هر قدم منطق را از view دورتر کرد. در ۲۰۲۶: **MVVM + UDF، یک `UiState` برای هر صفحه.**
- ViewModel از چرخش صفحه عبور می‌کند، **نه** از process death — کلیدها در `SavedStateHandle`.
- لایه‌ها: UI، سپس domain (اختیاری)، سپس data. repository منبع واحد حقیقت است.
- state به پایین، event به بالا. `StateFlow` منتشر کنید، هرگز `MutableStateFlow`.
- sealed interface برای حالت‌های جدا از هم؛ data class برای بخش‌هایی که کنار هم وجود دارند.
- eventهای یک‌باره: به شکل state مدلشان کنید و تأییدشان بگیرید؛ `SharedFlow(replay = 0)` ممکن است گمشان کند.
- MVI یعنی یک state، به‌علاوهٔ intentهای تایپ‌دار و یک reducer خالص — یک نظم، نه یک کتابخانه.
- use case فقط وقتی منطق چندجا استفاده می‌شود یا پیچیده است — بدون use case واسطهٔ خالی.
- ViewModel را با fake، `runTest` و یک rule برای Main dispatcher تست کنید.
- Clean Architecture: وابستگی‌ها **رو به داخل** — با ماژول‌های Gradle و Konsist در CI اعمالش کنید.

روز ۴ در راه است: Coroutines. این پست را ذخیره کنید و همراه ما باشید.



اسماعیل مرادی
@esmaeeil-moradi700